

HashCheck : Plagiarism Detection Tool

Phase - 1 Project Report

Team No. 12

GitHub : github.com/one7-himanshu/HashCheck-Plagiarism-Detection-and-Evaluation-Tool

Team Members -

1. Himanshu Bhardwaj [Leader]
2. Somil Bhatt
3. Shivam Saini
4. Sudarshan Juyal

Origin

The Problem: During our first year, we observed that students often copy code from a few sources i.e. ChatGPT, etc and just make changes in the variable to differentiate. Hence, making an unfair competition for others.

The Solution: Hence, we chose to make a CLI based multi-module program that will take two files, first, code/source data from AI tools and second, the file we need to check for plagiarism. The program will use the concept of Rolling Hash & K-Gram to reduce time and space complexity instead of the usual nested loop. Moreover, for easier evaluation, we will also add options to modify files before comparing.

Project Overview

We are developing a C++ based Plagiarism Checker designed to compare two source files and determine their similarity. To ensure efficient and accurate comparison, the system strips away formatting and comments, and utilizes a Rabin-Karp rolling hash algorithm alongside a hash table to compare chunks of code called k-grams. The project will help in evaluating large amounts of data by reducing time and space complexity as the Rabin-Karp Algorithm is used instead of the regular nested loop method. Moreover, the program will have the ability to check source files for balanced parenthesis, modify files before comparing using stack, etc.

System Execution Flow

To ensure efficient progress, we have divided the project into distinct modules. The program will

first read, modify and sanitize the raw input files. Next, it will define the mathematical logic required for hashing text chunks. After that, it will process the sanitized text through a similarity engine that compares the hashes. Finally, a central driver function will organise these steps and present the final output to the user.

Task Allocation & Implementation Strategy

1. File Handling and String Formation

Assigned to: Somil Bhatt

Somil is responsible for the initial data ingestion and preprocessing pipeline. His task is to access required files, modify if required and ensure that the core comparison engine only evaluates meaningful code characters, ignoring stylistic differences like spacing and comments.

- **Implementation Details :**

- **File Reading:** Somil will open and extract the contents of the provided .cpp file paths into a single continuous string.
- **Modification :** He will make necessary modification functions that can be required by the user in proper evaluation of the files.
- **Code Cleaning:** He will develop the cleanCode function, utilizing a while loop to iterate through the text. The logic will detect and skip single-line (//) and multi-line (/* */) comments, as well as strip out spaces, tabs (t), and newlines (n).
- **Standardization:** To ensure case differences do not bypass the plagiarism check, Somil will also write the convertToLowercase function, shifting all characters to lowercase using ASCII arithmetic.

2. Hashing System and Percentage Calculation

Assigned to: Himanshu Bhardwaj

Himanshu will build the mathematical foundation of the application. Instead of doing slow, character-by-character string comparisons, the system will use rolling hashes to assign unique numerical values to 10-character chunks called k-grams of code.

- **Implementation Details:**

- **Hash Algorithms:** Himanshu will implement the Rabin-Karp hashing functions: getHighestWeight, createInitialHash, and rollHash.
- **Mathematical Logic:** He will calculate hash values while preventing integer overflow using MOD and appropriate Base. His rollHash function will optimize performance by subtracting the outgoing character's weight and adding the new character's value, rather than recalculating the entire 10-character string.
- **Final Calculation:** Himanshu will calculate the final expression to determine the similarity score in percentage format along with an appropriate message.

3. Similarity Calculation Engine

Assigned to: Sudarshan Juyal

Sudharshan is tasked with building the core logical engine of the plagiarism checker. He will bridge the preprocessed strings with hashing math output to actively compare the two files and count the exact matches.

- **Implementation Details:**

- **Hash Table Integration:** Sudharshan will write the calculateSimilarity function. He will iterate through the first sanitized file, generating hashes for every 10-character k-gram, and store these hashes in a C++ hash table for efficient constant-time lookups.
- **Sliding Window Comparison:** He will then apply a queue sliding window approach over the second file. As the window moves one character at a time, he will generate the current hash and use the .count() method on the hash table to check if that hash exists in the first file.
- **Match Tracking:** Every time a match is found in the hash table, the logic will increment a match counter, which will eventually be passed to Himanshu's percentage formula.

4. Main Execution Flow

Assigned to: Shivam Saini

Shivam will serve as the system integrator. He is responsible for the program's primary menu driven interface, handling user input, and stringing together the functions built by the rest of the team into a cohesive, working application.

- **Implementation Details:**

- **User Interface:** Shivam will write the main() function to prompt the user for the local paths of the two .cpp files they wish to modify or/and compare.
- **Program Orchestration:** He will instantiate the PlagiarismChecker class object and sequentially trigger the workflow: calling Somil's read and clean functions, passing the cleaned strings to Himanshu's Mathematical Module then into Sudharshan's similarity engine, and executing the process.
- **Output Formatting:** At the end, He will use mapping to cleanly format the resulting float percentage returned by the program, displaying the final Plagiarism Score to the output console.